Study Material: Algorithmic Thinking with Python

B.Tech First Semester (S1) – Comprehensive Course Companion (2024 Scheme)

STUDENT NAME	DEPARTMENT	COURSE CODE & NAME
Anantha Krishnan B S	CSE (AI & ML)	EST 102 / UCEST105

MODULE 1: Problem-Solving Strategies & Computational Foundations

1.1 Fundamentals of Computational Problem Solving

A **computational problem** requires transforming specified input data into a desired output via a deterministic series of executable steps. The human approach to problem-solving relies on abstraction, decomposition, and model formulation.

The 6-Step Problem Solving Process in Computing

1. **Problem Understanding:** Precisely define the domain, constraints, inputs, and desired outputs.
2. **Formulating a Model:** Representing abstract concepts mathematically or logically (e.g., set theory, graphs).
3. **Developing an Algorithm:** Constructing a step-by-step, finite sequence of unambiguous instructions.
4. **Writing the Program:** Translating the algorithm into Python standard syntax.
5. **Testing & Debugging:** Validating correctness using corner test cases and eliminating logical errors.
6. **Evaluating Solution Efficiency:** Analyzing time and space complexity ($O(n)$ bounds).

1.2 Paradigms of Problem-Solving

Engineering problem solving demands selecting appropriate strategy paradigms based on domain complexity and computational resource boundaries:

- **Trial & Error Method:** An iterative search strategy where various options are attempted until success is achieved. *When to avoid:* High failure cost, safety-critical systems, or infinite search space.
- **Heuristics:** Practical "rules of thumb" or educated shortcuts that find satisfactory solutions quickly when optimal algorithms are computationally infeasible (e.g., A* pathfinding, Travelling Salesperson Problem).
- **Means-Ends Analysis:** An AI heuristic strategy that compares current state with goal state, identifies differences, and applies transformations (operators) to incrementally bridge the gap.
- **Backtracking:** Systematic search strategy that explores decision trees by extending candidate solutions and abandoning ("pruning") a path as soon as it determines it cannot yield a valid complete solution.

1.3 Classic Problem Solving Blueprint: River Crossing & Tower of Hanoi

Fox, Goose & Bag of Grain Problem

Rules: Boat capacity = Farmer + 1 item. Goose eats grain; Fox eats goose if left unattended.

Step Sequence:

1. Farmer takes Goose across (Leaves Fox & Grain).
2. Farmer returns alone.
3. Farmer takes Fox across.
4. Farmer returns bringing back Goose.
5. Farmer takes Grain across.
6. Farmer returns alone.
7. Farmer takes Goose across. Total: 7 trips.

Tower of Hanoi Logic:

Move n disks from Source (A) to Target (C) using Auxiliary (B).

```python
def hanoi(n, src, aux, tgt):
    if n == 1:
        print(f"Move disk 1: {src} -> {tgt}")
        return
    hanoi(n-1, src, tgt, aux)
    print(f"Move disk {n}: {src} -> {tgt}")
    hanoi(n-1, aux, src, tgt)
```

MODULE 1 (Contd.) & MODULE 2: Algorithms, Flowcharts & Python Essentials

1.4 Pseudocode & Flowchart Representation Standards

Pseudocode is an informal high-level description of an algorithm intended for human reading. It bypasses language syntax while maintaining strict logic sequencing, selection, and iteration.

Flowchart Symbol	Geometric Shape	Operational Function / Purpose
Terminal	Oval / Rounded Rectangle	Indicates the start or end point of an algorithm execution path.
Input / Output	Parallelogram	Denotes data entering the system or results displayed to user.
Process	Rectangle	Represents arithmetic operations, assignments, or state modifications.
Decision	Diamond	Evaluates conditional expressions ($True$ / $False$) for branching.
Connector	Circle	Joins different flowlines, especially across complex multi-path charts.

MODULE 2: Essentials of Python Programming

2.1 Variables, Dynamic Typing, and Numeric Data Types

Python utilizes **dynamic typing**—variables are untyped references to strongly typed objects in memory. Type checking occurs at runtime.

- `int`: Arbitrary-precision integers (e.g., `x = 100200300400500600`).
- `float`: Double-precision IEEE 754 64-bit floating point numbers (e.g., `pi = 3.14159`).
- `complex`: Numbers with real and imaginary parts (e.g., `z = 3 + 4j`).
- `bool`: Boolean values (`True` or `False`), subclass of integer where `True == 1`.

2.2 Python Operators & Precedence Rules

Operators evaluate according to operational precedence (PEMDAS / BEDMAS rules):

Operator Category	Operators	Associativity & Special Behavior
Exponentiation	`**`	Right-to-left associativity ($2 ** 3 ** 2 = 512$).
Arithmetic	`*`, `/`, `//`, `%`	`//` is floor division; `%` gives modulo remainder. Left-to-Right.
Additive	`+`, `-`	Standard addition/subtraction and string/list concatenation.
Relational / Comparison	`==`, `!=`, `>`, `<`, `>=`, `<=`	Supports chained comparisons (e.g., `1 < x < 10`).
Logical	`not`, `and`, `or`	Short-circuit evaluation applies. `not` > `and` > `or`.

2.3 Standard I/O & Format Specifiers

Python provides `input()` for receiving text (always returns string) and `print()` for output formatting.

```
# Format string (f-strings) and standard math library usage
import math

radius = float(input("Enter circle radius: "))
area = math.pi * (radius ** 2)
print(f"Radius: {radius:.2f} | Calculated Area: {area:.4f}")
```

KTU Exam Tip: Division Differences

In Python 3, single slash `5 / 2` performs float division giving `2.5`. Double slash `5 // 2` performs floor (integer) division returning `2`. Modulo `5 % 2` yields remainder `1`.

MODULE 3: Selection Statements & Iteration Control Structures

3.1 Selection Constructs: Conditional Branching

Conditional structures alter program execution flow based on boolean expressions. Indentation (4 spaces) defines block scoping instead of curly braces.

```
# Nested & Elif Selection Structure: Marks Grading System
mark = int(input("Enter KTU Exam Score (0-100): "))

if mark >= 90:
    grade = "S (Outstanding)"
elif mark >= 80:
    grade = "A+ (Excellent)"
elif mark >= 70:
    grade = "B (Above Average)"
elif mark >= 45:
    grade = "P (Pass)"
else:
    grade = "F (Failed)"
print(f"Final Grade Result: {grade}")
```

3.2 Iteration Constructs: For Loops, While Loops & Range Function

Python provides two main looping constructs: `for` (definite iteration over sequences) and `while` (indefinite iteration based on condition).

The `range()` function: Generates arithmetic progressions: `range(start, stop[, step])`. Note that `stop` is exclusive!

Prime Number Check (For Loop)

```
num = 29
is_prime = True
for i in range(2, int(math.sqrt(num)) + 1):
    if num % i == 0:
        is_prime = False
        break
print(f"{num} Prime? {is_prime}")
```

GCD via Euclidean (While Loop)

```
a, b = 56, 98
while b != 0:
    a, b = b, a % b
print(f"GCD is: {a}") # Outputs 14
```

3.3 Loop Control Statements & Else Clause on Loops

Python supports explicit control flow adjustments within iteration loops:

- `break`: Immediately terminates the innermost loop execution.
- `continue`: Skips the remaining statement block in current iteration and jumps to the next iteration.
- `pass`: Null statement used as a syntactical placeholder.
- `loop...else` block: Executes ONLY if the loop completes normally (without encountering a `break` statement).

Classic Problem: Star Pattern Construction using Nested Loops

```
n = 4
for i in range(1, n + 1):
    # Print space padding followed by stars
    print(" " * (n - i) + "*" * (2 * i - 1))
```

MODULE 3 (Contd.): Built-in Sequence Data Types & NumPy Arrays

3.4 Core Sequence Data Structures Comparison

Python contains powerful built-in collection types categorized by mutability and ordering characteristics:

Data Structure	Mutable?	Ordered?	Duplicates?	Syntax & Primary Use Case
List	Yes	Yes	Allowed	`[1, 2, "AI"]` - General dynamic arrays, stack/queue.
Tuple	No	Yes	Allowed	`(1, 2, "ML")` - Immutable records, function return pairs.
Set	Yes	No	Unique Only	`{1, 2, 3}` - Mathematical set operations, deduplication.
Dictionary	Yes	Key-Preserved	Keys Unique	`{"id": 101, "dept": "AIML"}` - Hash map key-value lookup.
String	No	Yes	Allowed	`"TKMIT"` - Sequence of Unicode characters.

3.5 Operations on Sequences & List Comprehensions

List Slicing Syntax: `sequence[start:stop:step]` (negative step reverses sequence).

```
# Slicing & Dictionary / List Manipulation
numbers = [10, 20, 30, 40, 50, 60]
reversed_lst = numbers[::-1] # [60, 50, 40, 30, 20, 10]

# List Comprehension: [expression for item in iterable if condition]
evens_squared = [x**2 for x in range(10) if x % 2 == 0] # [0, 4, 16, 36, 64]

# Dictionary Operations
student = {"name": "Anantha", "college": "TKMIT"}
student["grade"] = "S"
for key, val in student.items():
    print(f"{key} -> {val}")
```

3.6 Arrays in Python via NumPy Library

While native lists store heterogeneous element references, standard machine learning demands homogeneous contiguous memory arrays via the **NumPy** library.

```
import numpy as np

# Creating 1D and 2D arrays
arr1d = [np.array](http://np.array)([1, 2, 3, 4, 5])
matrix = [np.array](http://np.array)([[1, 2], [3, 4]])

# Vectorized Arithmetic Operations (Element-wise)
doubled = arr1d * 2  # [2, 4, 6, 8, 10]
dot_product = np.dot(arr1d, arr1d) # 55
print(f"Shape: {matrix.shape} | Mean: {np.mean(arr1d)}")
```

MODULE 4: Decomposition, Modularization & Function Architecture

4.1 Problem Decomposition & Modularization Strategy

Decomposition breaks complex engineering software down into manageable, independent sub-problems. **Modularization** packages related functionality into standalone functions or file modules, driving code reuse and maintainability.

4.2 Function Definition, Arguments, and Multiple Return Values

Functions are declared using the `def` keyword. Arguments support positional values, default parameters, variable keyword arguments, and multiple returns returned as a tuple.

```
# Comprehensive Function Example with Multiple Return Values
def compute_circle_metrics(radius=1.0):
    """Calculates and returns area and perimeter of a circle."""
    area = math.pi * (radius ** 2)
    perimeter = 2 * math.pi * radius
    return area, perimeter  # Unpacks as a 2-element tuple

# Caller site unpacking
a, p = compute_circle_metrics(5.0)
print(f"Area: {a:.2f}, Perimeter: {p:.2f}")
```

4.3 Variable Scope & LEGB Rule

Python resolves variable references using the **LEGB Rule** hierarchy:

- **L (Local):** Names assigned within a function body.
- **E (Enclosing):** Names in local scopes of enclosing/nested functions.
- **G (Global):** Names assigned at top level of module file.
- **B (Built-in):** Preassigned names in Python (e.g., `len`, `range`, `ValueError`).

```
counter = 10 # Global variable

def increment_counter():
    global counter # Required to modify global scope reference
    counter += 1

increment_counter() # counter becomes 11
```

4.4 Custom Modules & Importing Constructs

Any standard Python file (`.py`) functions as a module. Specific capabilities are imported using standard statements.

KTU Syllabus Program: Custom Module Import

File 1 (`fibomodule.py`):

```
def get_fibonacci(n):
    a, b = 0, 1
    seq = []
    while len(seq) < n:
        seq.append(a)
        a, b = b, a + b
    return seq
```

File 2 (`main.py`): `from fibomodule import get_fibonacci; print(get_fibonacci(7))`

MODULE 4 (Contd.): Recursion, Call Stack Analysis & Core Algorithms

4.5 Recursion Mechanics & The Call Stack

Recursion is a method where a function calls itself to solve smaller instances of the same problem. Every recursive implementation requires:

- **Base Case:** Stopping condition that returns a value without further recursive calls (prevents infinite loops).
- **Recursive Step:** Reduces the problem size toward the base case.

Call Stack Trace Visualization for Factorial(3)

Each function call pushes a frame onto the system call stack. Stack pops upon reaching base case:

```
[Stack Push Phase]                      [Stack Pop / Unwinding Phase]
fact(3) -> 3 * fact(2)                  fact(1) returns 1
  fact(2) -> 2 * fact(1)                fact(2) returns 2 * 1 = 2
    fact(1) -> returns 1 (Base Case)    fact(3) returns 3 * 2 = 6
```

4.6 Classic KTU Recursive Implementations

1. Greatest Common Divisor (GCD)

```python
def gcd_rec(a, b):
    if b == 0:
        return a
    return gcd_rec(b, a % b)
```

2. N-th Fibonacci Number

```python
def fibo_rec(n):
    if n <= 1:
        return n
    return fibo_rec(n-1) + fibo_rec(n-2)
```

3. Sum of Digits of Positive Integer

```python
def sum_digits(n):
    if n == 0:
        return 0
    return (n % 10) + sum_digits(n // 10)
```

4. Sum of Array Elements

```python
def array_sum(arr):
    if len(arr) == 0:
        return 0
    return arr[0] + array_sum(arr[1:])
```

4.7 Practical Validation Example: Indian Mobile Number Verification

```python
def validate_mobile_number(num_str):
    """Rules: Exactly 10 digits; Starts with 7, 8, or 9."""
    if len(num_str) == 10 and num_str.isdigit() and num_str[0] in ['7', '8', '9']:
        return True
    return False

print(f"Is 9847012345 Valid? {validate_mobile_number('9847012345')}") # True
```

MODULE 5: Advanced Computational Approaches & Paradigms

5.1 Survey of Algorithmic Paradigms

Selecting the optimal paradigm depends on problem structure, subproblem overlapping properties, and runtime constraints.

Paradigm	Core Principle / Methodology	Representative Algorithm / Example
Brute-Force	Exhaustive search evaluating all possible candidate solutions.	Password guessing, Padlock combinations, Linear Search.
Divide & Conquer	Break problem into non-overlapping subproblems, solve recursively, combine.	Merge Sort, Quick Sort, Binary Search.
Dynamic Programming	Solve overlapping subproblems once and store results (Memoization/Tabulation).	0/1 Knapsack, Memoized Fibonacci, Longest Common Subsequence.
Randomized Algorithm	Employs random number generator bits to influence logic behavior.	Coupon Collector Problem, Randomized QuickSelect.

5.2 Divide & Conquer Paradigm: Merge Sort Algorithm

Merge sort divides input list into two equal halves, recursively calls itself on both, and merges the sorted halves.

```python
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    # Merge step
    merged, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i]); i += 1
        else:
            merged.append(right[j]); j += 1
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged
```

5.3 Dynamic Programming (DP) vs Pure Recursion

Pure Recursion ($O(2^n)$ Time)

Recomputes identical Fibonacci sub-branches repeatedly, causing stack overflow for large n.

DP with Memoization ($O(n)$ Time)

```python
def fibo_dp(n, memo={}):
    if n in memo: return memo[n]
    if n <= 1: return n
    memo[n] = fibo_dp(n-1, memo) +
fibo_dp(n-2, memo)
    return memo[n]
```

MODULE 5 (Contd.) & KTU University Exam Review Kit

5.4 File Handling & Exception Handling Mechanics

Python provides safe file stream management via the `with` context manager, automatically closing files upon scope exit.

```python
# Safe File Reading & Exception Handling
try:
    with open("sample_data.txt", "r") as file:
        lines = file.readlines()
        print(f"Total Lines Processed: {len(lines)}")
except FileNotFoundError:
    print("Error: Specified file does not exist in path!")
except Exception as e:
    print(f"An unexpected error occurred: {e}")
```

5.5 Core Exception Architecture

Exceptions are handled using `try`, `except`, `else`, and `finally` blocks:

- `try`: Encloses code that might throw an exception.
- `except`: Catches and executes handling logic for specific exception types.
- `else`: Executes if *no* exception was raised in the try block.
- `finally`: Always executes regardless of exception state (ideal for cleanups).

KTU S1 CSE AIML - Final Exam Revision & Question Kit

Must-Practice University Exam Questions (10-Mark Standards)

1. **Module 1:** Explain Means-Ends Analysis and Backtracking with real-world examples. Solve the Farmer, Fox, Goose & Grain problem step-by-step.
2. **Module 2:** Explain operator precedence in Python with an example expression. Write a program to calculate compound interest using the `math` module.
3. **Module 3:** Write a Python program using nested loops to print prime numbers below 100. Demonstrate array creation using NumPy.
4. **Module 4:** Write recursive functions for: (a) Finding GCD of two numbers, (b) Generating n Fibonacci terms, and (c) Summing digits of a number. Trace the call stack for $GCD(48, 18)$.
5. **Module 5:** Compare Brute-Force, Divide & Conquer, and Dynamic Programming. Write a Python program to implement Merge Sort on an array of integers.

Student Summary & Verification Note

This study material has been compiled for **Anantha Krishnan B S**, S1 B.Tech CSE (AI & ML), **TKM Institute of Technology (TKMIT)** under the APJ Abdul Kalam Technological University (KTU) 2024 Scheme syllabus for *Algorithmic Thinking with Python* (EST 102 / UCEST105).